

# ROOM EN ANDROID: TEORÍA Y CONCEPTOS FUNDAMENTALES

---

## *Persistencia de datos con Kotlin, Room y SQLite*

Este documento presenta los conceptos necesarios para comprender y utilizar Room, la biblioteca de persistencia de Android basada en SQLite. El objetivo es entender qué problema resuelve Room, cómo se estructura una aplicación con Room y cómo se relacionan las entidades, los DAO, la base de datos y la aplicación.

### 1. ¿Qué es Room?

Room es una biblioteca de Jetpack que proporciona una capa de abstracción sobre SQLite. Permite trabajar con bases de datos locales desde Kotlin utilizando clases, interfaces, anotaciones y consultas SQL, reduciendo parte del código repetitivo y facilitando la detección de errores durante la compilación.

Room no sustituye a SQLite como motor de almacenamiento. SQLite continúa siendo el sistema que almacena físicamente los datos. Room se sitúa entre la aplicación y SQLite para proporcionar una forma estructurada de acceder a esos datos.

### 2. Room frente a SQLite

| Concepto               | SQLite                       | Room                                              |
|------------------------|------------------------------|---------------------------------------------------|
| Motor de base de datos | Sí                           | Utiliza SQLite                                    |
| Tablas                 | Se definen mediante SQL      | Se representan mediante @Entity                   |
| Operaciones            | SQL directo                  | DAO + anotaciones + SQL cuando sea necesario      |
| Objetos Kotlin         | Conversión manual            | Mapeo entre entidades y objetos                   |
| Errores de consultas   | Pueden aparecer en ejecución | Room puede detectar muchos errores en compilación |
| Integración Android    | Más manual                   | Diseñada específicamente para Android/Jetpack     |

### 3. Arquitectura básica de Room

Una aplicación sencilla con Room se organiza alrededor de tres componentes principales:

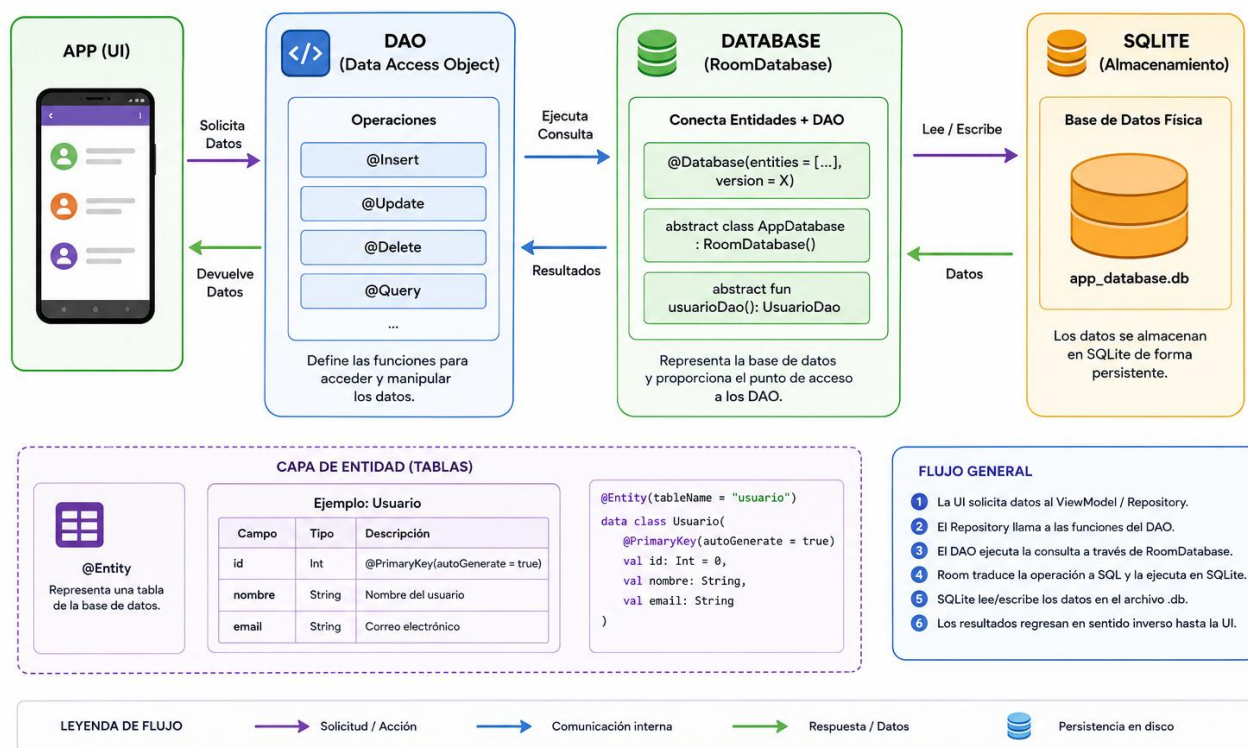
- Entity (@Entity): representa una tabla de la base de datos.
- DAO (@Dao): define las operaciones para consultar y modificar los datos.
- RoomDatabase (@Database): representa la base de datos y proporciona acceso a los DAO.

El flujo conceptual puede resumirse así:

**APP → DAO → RoomDatabase → SQLite → RoomDatabase → DAO → APP**

# ESQUEMA DE FLUJO DE ROOM (Android)

De tu app → Room → SQLite → Datos persistentes



## 4. Conceptos básicos de bases de datos

**Base de datos:** Conjunto organizado de datos almacenados de forma persistente.

**Tabla:** Estructura que almacena registros de un determinado tipo.

**Registro o fila:** Una instancia concreta de los datos almacenados en una tabla.

**Campo o columna:** Una propiedad o dato que forma parte de cada registro.

**Clave primaria:** Campo que identifica de forma única cada registro.

**Clave foránea:** Campo que permite relacionar una tabla con otra.

**Consulta:** Operación utilizada para obtener o manipular información.

**CRUD:** Create, Read, Update, Delete: crear, leer, actualizar y eliminar.

## 5. Entity: representar una tabla

Una Entity es una clase Kotlin marcada con @Entity. Room utiliza esta clase como modelo de una tabla. Sus propiedades normalmente representan las columnas.

Ejemplo conceptual:

```
@Entity(tableName = "usuario")
data class Usuario(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val nombre: String,
    val email: String
)
```

## 6. @Entity

@Entity indica a Room que una clase debe convertirse en una tabla. El parámetro tableName permite especificar el nombre de la tabla. Si no se indica, Room puede utilizar el nombre de la clase según la configuración.

## 7. @PrimaryKey

@PrimaryKey identifica la columna que actúa como clave primaria. Con autoGenerate = true, Room puede generar automáticamente valores para la clave en cada nuevo registro.

## 8. DAO: Data Access Object

DAO significa Data Access Object. Es el componente encargado de definir cómo la aplicación accede a los datos. En Room normalmente se declara como una interface marcada con @Dao.

Ejemplo:

```
@Dao
interface UsuarioDao {

    @Insert
    suspend fun insertar(usuario: Usuario)

    @Query("SELECT * FROM usuario")
    suspend fun obtenerTodos(): List<Usuario>

    @Update
    suspend fun actualizar(usuario: Usuario)

    @Delete
    suspend fun eliminar(usuario: Usuario)
}
```

## 9. Operaciones CRUD en Room

| Operación | Room    | Función                                 |
|-----------|---------|-----------------------------------------|
| Create    | @Insert | Introduce nuevos registros.             |
| Read      | @Query  | Obtiene información mediante consultas. |
| Update    | @Update | Modifica registros existentes.          |
| Delete    | @Delete | Elimina registros.                      |

## 10. @Query y SQL

@Query permite escribir consultas SQL directamente en el DAO. Esto resulta especialmente útil para búsquedas, filtros, ordenaciones, relaciones y consultas que no se resuelven simplemente con @Insert, @Update o @Delete.

```
@Query("SELECT * FROM usuario WHERE nombre = :nombre")
suspend fun buscarPorNombre(nombre: String): List<Usuario>
```

El parámetro :nombre se vincula con el parámetro nombre de la función. Room valida la consulta y realiza el mapeo del resultado hacia el tipo indicado.

## 11. RoomDatabase

La clase que hereda de RoomDatabase representa la base de datos de la aplicación. Se marca con @Database y especifica las entidades que forman parte de ella y su versión.

```
@Database(
    entities = [Usuario::class],
    version = 1
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun usuarioDao(): UsuarioDao
}
```

## 12. ¿Qué significa version?

La versión identifica la estructura de la base de datos. Cuando se modifica una Entity o la estructura de las tablas, normalmente hay que aumentar la versión y definir una migración para actualizar las bases de datos existentes.

## 13. Creación de la base de datos

Una instancia de RoomDatabase se suele construir con Room.databaseBuilder(). En una aplicación Android real conviene controlar cuidadosamente la creación de esta instancia para evitar crear múltiples conexiones innecesarias.

```
val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java,
    "app_database"
).build()

val usuarioDao = db.usuarioDao()
```

## 14. Corrutinas y suspend

Las operaciones de base de datos pueden tardar y no deben bloquear el hilo principal. Por ello es habitual utilizar funciones suspend y corrutinas de Kotlin para ejecutar operaciones de Room de manera asíncrona.

La palabra suspend indica que una función puede suspenderse y reanudarse dentro de una corrutina. No significa por sí misma que cree un hilo nuevo; la ejecución debe realizarse en el contexto apropiado.

## 15. Flow y observación de datos

Room puede devolver datos mediante Flow de Kotlin. Esto permite observar cambios y actualizar la interfaz de usuario cuando los datos de la base de datos cambian.

```
@Query("SELECT * FROM usuario")
fun observarUsuarios(): Flow<List<Usuario>>
```

## 16. Repository: una capa recomendada

Aunque Room puede utilizarse directamente desde otras capas de la aplicación, una arquitectura habitual añade un Repository. Su función es centralizar el acceso a los datos y evitar que la interfaz tenga que conocer directamente los detalles de Room.

**Flujo recomendado:** UI → ViewModel → Repository → DAO → Room → SQLite

## 17. ViewModel

El ViewModel mantiene y gestiona el estado relacionado con la interfaz de usuario. En una arquitectura moderna puede solicitar datos al Repository y exponerlos mediante StateFlow, Flow u otros mecanismos de estado.

## 18. Relaciones entre tablas

Las aplicaciones reales suelen necesitar varias tablas relacionadas. Room permite modelar relaciones como uno a uno, uno a muchos y muchos a muchos.

- Uno a uno: un registro de una tabla se relaciona con un registro de otra.
- Uno a muchos: un registro puede tener varios registros relacionados.
- Muchos a muchos: varios registros de una tabla pueden relacionarse con varios de otra.

## 19. @ForeignKey

@ForeignKey permite establecer restricciones de integridad referencial entre tablas. Por ejemplo, una tabla Pedido puede contener el id del Usuario al que pertenece.

## 20. @Relation

@Relation permite definir cómo Room debe representar determinados datos relacionados en objetos Kotlin. Es especialmente útil cuando una consulta necesita devolver una estructura formada por datos de varias tablas.

## 21. Migraciones

Una migración es el proceso mediante el cual se actualiza una base de datos existente cuando cambia su estructura. Por ejemplo, si una aplicación pasa de la versión 1 a la versión 2 y se añade una nueva columna, Room necesita conocer cómo realizar ese cambio.

```
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "ALTER TABLE usuario ADD COLUMN telefono TEXT"
        )
    }
}
```

```

    }
}

```

## 22. Anotaciones importantes de Room

| Anotación   | Función                                                                           |
|-------------|-----------------------------------------------------------------------------------|
| @Entity     | Define una entidad/tabla.                                                         |
| @PrimaryKey | Define la clave primaria.                                                         |
| @Dao        | Define un objeto de acceso a datos.                                               |
| @Insert     | Inserta datos.                                                                    |
| @Update     | Actualiza datos.                                                                  |
| @Delete     | Elimina datos.                                                                    |
| @Query      | Ejecuta una consulta SQL.                                                         |
| @Database   | Define la base de datos Room.                                                     |
| @ForeignKey | Define una clave foránea.                                                         |
| @Relation   | Modela relaciones entre datos.                                                    |
| @Ignore     | Indica que un campo o constructor no debe ser persistido por Room.                |
| @ColumnInfo | Permite personalizar información de una columna.                                  |
| @Embedded   | Permite incluir las propiedades de otro objeto dentro de una entidad o resultado. |

## 23. Flujo completo de una operación

1. La interfaz de la aplicación solicita una operación.
2. El ViewModel coordina la acción.
3. El Repository solicita los datos al DAO.
4. El DAO ejecuta una operación definida mediante una anotación o una consulta SQL.
5. Room traduce la operación al acceso correspondiente sobre SQLite.
6. SQLite lee o modifica los datos almacenados.
7. Room convierte los resultados al tipo Kotlin correspondiente.
8. El resultado vuelve al DAO, Repository y ViewModel.
9. La interfaz observa el nuevo estado y lo muestra al usuario.

## 24. Ejemplo completo mínimo

Entidad:

```

@Entity
data class Usuario(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val nombre: String,
    val email: String
)

```

DAO:

```
@Dao
interface UsuarioDao {

    @Insert
    suspend fun insertar(usuario: Usuario)

    @Query("SELECT * FROM Usuario")
    fun obtenerUsuarios(): Flow<List<Usuario>>
}
```

Base de datos:

```
@Database(entities = [Usuario::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun usuarioDao(): UsuarioDao
}
```

## 25. Errores y conceptos que conviene evitar

| Error                                           | Solución / concepto                                                          |
|-------------------------------------------------|------------------------------------------------------------------------------|
| Confundir Room con SQLite                       | Room es una capa de persistencia; SQLite es el motor que almacena los datos. |
| Hacer operaciones pesadas en el hilo principal  | Utiliza corrutinas y el contexto adecuado.                                   |
| Olvidar la clave primaria                       | Una Entity necesita una identificación adecuada.                             |
| Cambiar la Entity sin controlar la versión      | Los cambios de esquema requieren gestionar migraciones.                      |
| Meter toda la lógica en el DAO                  | En proyectos medianos/grandes es preferible separar responsabilidades.       |
| Crear demasiadas instancias de la base de datos | Normalmente se utiliza una instancia controlada/reutilizable.                |

## 26. Conceptos que debes dominar

- SQLite y bases de datos relacionales.
- Tablas, filas, columnas y claves.
- CRUD.
- Kotlin data class.
- Anotaciones de Kotlin/Room.
- @Entity y @PrimaryKey.
- @Dao, @Query, @Insert, @Update y @Delete.
- RoomDatabase y databaseBuilder.
- Corrutinas y suspend.
- Flow / StateFlow.
- Repository y ViewModel.
- Relaciones entre entidades.
- Claves foráneas.
- Migraciones y versiones.

## 27. Resumen final

La idea esencial que debes recordar es que Room organiza el acceso a una base de datos SQLite mediante objetos Kotlin. La Entity representa la tabla, el DAO define las operaciones, RoomDatabase conecta los componentes y SQLite realiza el almacenamiento persistente.

**ENTITY → DAO → ROOMDATABASE → SQLITE**

En una arquitectura Android más completa, el flujo habitual es: UI → ViewModel → Repository → DAO → Room → SQLite, y los datos regresan hacia la UI mediante Flow o StateFlow.